Deception Benchmark: A Stress-Test Benchmark for Trusted Vulnerability Detection with Honeypot Anti-Overfitting

Anshumali Shrivastava*†‡, Neha Rungta*, Alexander Greaves-Tunnell*
*Amazon Web Services †Rice University ‡Corresponding author: anshumsh@amazon.com

***Abstract*—AI-powered security tools achieve near-perfect vulnerability recall but generate false positive rates of 52–99% on code that security engineers routinely classify correctly. This is not merely a production inconvenience: it reveals that models detect vulnerability patterns without genuinely understanding code. You cannot compensate for this with a harness; as attacks self-improve, the attack surface will outrun any static verification layer. Yet no existing benchmark measures this: current evaluations test whether models can *detect* known vulnerabilities, not whether they can tell real threats from safe code that merely looks suspicious.**

We introduce Deception Benchmark, a 14,822-sample benchmark spanning 16 programming languages and 70+ CWE categories, specifically designed to stress-test discrimination capability. Safe samples exhibit genuine vulnerability patterns with subtle mitigations that break the exploit chain. We evaluate 13 models from 6 providers under two prompting strategies and find that no model achieves both FPR $< 10\%$ and FNR $< 10\%$ simultaneously, a very generous threshold to be interesting for production use.

We additionally introduce *Honeypot Benchmarking*, a novel anti-overfitting methodology that embeds deliberately ambiguous samples to structurally resist benchmark gaming. We provide a game-theoretic argument for its optimality and discuss its applicability to ML benchmarking broadly.

***Index Terms*—Security benchmarks, vulnerability detection, false positives, LLM evaluation, benchmark integrity**

I. Introduction

The deployment of AI agents for security work (vulnerability triage, code review, cloud misconfiguration analysis, and incident response) has progressed from research demonstrations to production adoption. Orchestration and coding agents have shown remarkable capabilities: given a hypothesis, they can execute multi-step verification chains, interact with tools, and reason across complex codebases. Large language models are particularly adept at *sensing* potential security issues, identifying suspicious patterns, recognizing vulnerability idioms, and generating plausible exploit scenarios. However, sensing a potential vulnerability is not the same as understanding the code. The industry's current focus on agent harnesses (multi-step verification loops that compensate for model limitations) addresses symptoms rather than the underlying capability gap. A model that does not intrinsically understand why a mitigation holds will eventually be outpaced by self-improving attacks regardless of what harness surrounds it. Measuring this intrinsic understanding is what Deception Benchmark is designed to do.

The cost of this gap is asymmetric. In offensive security or CTF contexts, false positives have low cost: an agent that generates many hypotheses and validates a few working exploits is valuable. In defensive security, the economics are inverted. A single false critical finding triggers incident protocols, blocks deployments, and demands immediate human investigation, often consuming senior engineering hours to resolve. After a handful of false alarms, security teams lose trust in the tool entirely, negating its value regardless of detection capability.

This asymmetry is compounded by the nature of real-world security code. Unlike laboratory benchmarks where vulnerable and safe code look obviously different, production code frequently contains patterns that *appear* dangerous (defensive code that handles sensitive data, implements security boundaries, or processes untrusted input) but includes mitigations that prevent exploitation. A competent security engineer resolves these distinctions routinely. Current AI systems cannot.

The problem is not unique to any provider or model family. We evaluate 13 frontier models from 6 providers (Anthropic, OpenAI, Meta, Amazon, Mistral, DeepSeek) and find that **every model exhibits the same failure mode**: near-perfect recall paired with unacceptable false positive rates. Moreover, the standard approach of requiring models to produce concrete exploits before flagging (Proof-of-Exploit prompting) reduces false positives but introduces unacceptable false negatives. The fundamental tradeoff remains unsolved.

Existing security benchmarks (CyberGym [1], CyberSecEval [2], CYBENCH [3], VulnBench [4]) measure capability ("can it find a vulnerability that exists?") but not precision on hard negatives ("can it correctly identify safe code that looks vulnerable?"). None contains difficult safe samples designed to stress-test discrimination.

Moreover, the distribution of code is shifting. Just as BrowseComp [14] recognized that future search queries will be agent-generated and require benchmarks calibrated to agent-level complexity, we observe that code analyzed by security tools is increasingly agent-written. LLMs produce complex, multi-layered code at scale. The security challenges of tomorrow will be subtle interactions across abstraction layers, race conditions gated by infrastructure configuration, and mitigations that require deep contextual reasoning to verify. A benchmark that current models solve easily measures nothing useful about future readiness. Deception Benchmark is designed to stress-test the level of understanding that frontier

models need to be genuinely trustworthy as security agents.

Recent work on automated red-teaming reinforces this direction. OpenAI's GPT-Red [15] demonstrates that models trained through self-play adversarial loops discover attack patterns that static frontier models cannot defend against. Attack complexity grows through self-improvement faster than static defenses can adapt. The implication for defensive AI is direct: a system that relies on a static verification harness around a model that does not intrinsically understand the code will be outpaced as attacks evolve. Current long-horizon benchmarks reward eventual success regardless of how many attempts it takes: a model that brute-forces through 100 verification loops scores the same as one that understands the code on the first pass. But in production security, each attempt has real cost, as every finding triggers human review. What matters is whether the model can get the judgment right without trial-and-error. Deception Benchmark measures this: given unlimited reasoning time but no external verification tools, does the model genuinely understand why code is safe?

A. Contributions

1) **Deception Benchmark**: A 14,822-sample benchmark (16 languages, 70+ CWEs) with two challenge types (code-level and environment-gated) where safe samples exhibit real vulnerability patterns with subtle mitigations. To our knowledge, this is the largest security vulnerability detection benchmark with calibrated hard negatives, at over 10× the scale of prior work.
2) **Comprehensive cross-provider evaluation**: 13 models from 6 providers under two prompting strategies (Direct and Proof-of-Exploit), demonstrating that the FPR/FNR tradeoff is fundamental and provider-independent.
3) **Honeypot Benchmarking**: A novel anti-overfitting methodology that embeds deliberately ambiguous samples to structurally resist benchmark gaming, with a game-theoretic argument for its optimality.
4) **Environment-gated reasoning**: A challenge type that requires models to reason about whether deployment infrastructure (network policies, WAFs, connection pooling, IAM boundaries) neutralizes a code-level vulnerability, measuring the kind of contextual reasoning that production security demands.

II. Related Work

A. Security Evaluation Benchmarks

CyberGym [1] evaluates agents on 1,507 realistic security tasks spanning offensive and defensive operations. CyberSecEval [2] measures exploit generation capability across buffer overflows and memory corruption. CYBENCH [3] tests 40 professional-level CTF challenges. SEC-Bench [5] targets authentic security engineering workflows. VulnBench [4] addresses methodological issues in vulnerability detection evaluation.

All measure *recall*: the ability to detect or exploit vulnerabilities that exist. None systematically evaluates *precision* on hard negatives, i.e., code that exhibits vulnerability patterns but is not exploitable. This gap is critical because the recall problem is largely solved (95–100% across all frontier models) while the precision problem remains far from production-ready (FPR 74–98% with standard prompting).

B. Constructive Verification

Proof-of-Exploit (PoE) approaches require demonstrating exploitability before asserting vulnerability. White et al. [6] propose cryptographically verified LLM security evaluation. Green et al. [7] develop toolchains for real-world exploit proof. Xue et al. [8] apply PoC verification before alerting to reduce false positives.

We adopt PoE as an evaluation protocol, not a contribution in itself, but a methodology that gives models the best possible chance at precision and establishes the upper bound of what current prompting approaches can achieve.

C. Benchmark Contamination and Gaming

Benchmark degradation through contamination, overfitting, and gaming is well-documented [9], [10]. Defenses include canary strings, rotating holdouts, and membership inference detection. SQuAD 2.0 [11] introduced unanswerable questions as an empirical design choice. NPHardEval [12] proposes monthly benchmark refresh.

No prior work formalizes the game-theoretic properties of embedding unsolvable items as a structural defense. Our Honeypot Benchmarking methodology provides this formalization.

III. Dataset Construction

A. Design Philosophy

Deception Benchmark measures *discrimination*: the ability to distinguish exploitable vulnerabilities from safe code that exhibits the same patterns. The key insight is that in production security, the difficulty is not recognizing vulnerability patterns (models already excel at this) but verifying whether a specific mitigation breaks the exploit chain.

Each sample presents code that contains a recognizable security-relevant pattern. The benchmark is balanced: 50% of samples are genuinely exploitable, 50% contain the same patterns but with mitigations that prevent exploitation. Models that rely on pattern recognition without deep reasoning about mitigation effectiveness will achieve approximately 50% accuracy, equivalent to random guessing.

B. Challenge Types

Code-level challenges. Each sample contains source code in one of 16 programming languages. Vulnerable and safe variants are constructed to be superficially similar: both exhibit the same suspicious patterns (unsanitized inputs, dangerous function calls, race-prone access patterns), but safe variants include subtle fixes (type assertions, bounds checks, sanitization steps, proper synchronization) that neutralize the exploit path.

Environment-gated challenges. The same code is presented with a deployment environment description. The environment may contain infrastructure controls (Kubernetes

NetworkPolicies, PgBouncer connection pooling configurations, AWS WAF rules, IAM permission boundaries, seccomp profiles) that completely block the exploit path despite the vulnerability pattern being visible in the source code. These challenges require reasoning about whether a vulnerability is exploitable *in a specific deployment context*, not merely whether the pattern exists in isolation.

C. Generation and Curation

Samples are produced through a pipeline involving heavy use of LLMs with human-in-the-loop oversight, drawing on real codebases and public security datasets for grounding. Samples are iteratively refined to ensure difficulty against state-of-the-art models. The resulting code uses real frameworks (Flask, Spring Boot, Express, Go net/http), real CWE categories, and realistic software architecture patterns (microservices, message queues, CI/CD pipelines, database migrations).

Quality curation involved multi-model consensus evaluation: each sample was evaluated by multiple frontier models across providers. Samples where the gold-standard label was disputed by unanimous model consensus underwent independent adjudication. This process identified and removed 2,096 samples with incorrect gold-standard labels and retained 1,484 samples as honeypots (see Section IV).

All samples are originally authored, not extracted from any real codebase, eliminating IP concerns and ensuring no training-data contamination. The code is structurally realistic (50–200 lines per sample) with proper imports, error handling, and naming conventions.

D. Dataset Statistics

TABLE I: Deception Benchmark Dataset Statistics

Metric	Value
Total public samples	14,822
Scored samples	11,478
Honeypot samples	3,344
Balance (scored)	50% / 50%
Languages	16
CWE categories	70+
Code-level challenges	7,712
Environment-gated challenges	3,766
Avg. sample length	120 lines

IV. Honeypot Benchmarking

A. Motivation

Benchmarks degrade over time. Public answer keys enable data contamination during training [9]. Fixed test sets enable overfitting through repeated submission and score observation [10]. Within months of publication, a benchmark may measure exposure to the test rather than genuine capability. Current defenses (held-out test sets, rotating challenges, canary strings) are reactive. We propose a *structural* defense.

B. Design

Honeypot Benchmarking embeds deliberately ambiguous samples within the public benchmark. These are cases where expert adjudication across multiple frontier models could not reach consensus on the correct label. The samples are:

- **Not scored**: they do not affect the benchmark result.
- **Indistinguishable**: they cannot be differentiated from scored samples without the secret partition key.
- **Maximally hard**: they represent the frontier of legitimate ambiguity, not random noise.
- **Attractive**: optimization processes naturally target the samples they fail on most consistently, which are precisely the honeypots.

Like honeypots in network security [13], these samples attract and trap optimization effort, wasting attacker resources while revealing gaming attempts.

C. Game-Theoretic Analysis

We model benchmark gaming as a two-player game between an *evaluator* (benchmark designer) and an *optimizer* (entity seeking to maximize reported score).

Setup. The benchmark B consists of N scored items and K honeypot items, shuffled uniformly. The evaluator knows the partition $\pi : B \to \{scored, honeypot\}$; the optimizer does not.

Optimizer's observation. The optimizer submits predictions and observes only the aggregate score $S = \frac{\text{correct}(\pi^{-1}(scored))}{N}$. It cannot decompose its performance into scored and honeypot components.

Information-theoretic bound. To identify which K of $N + K$ items are honeypots requires searching over $\binom{N+K}{K}$ possible partitions. For Deception Benchmark ($N = 11{,}478$, $K = 3{,}344$), this is $\binom{14822}{3344} \approx 10^{4200}$, computationally intractable even with unlimited submission attempts.

Wasted effort. Any optimization process that targets the hardest failures (standard behavior for gradient-based or prompt-search optimization) will allocate effort proportional to sample difficulty. Since honeypots are by construction the hardest samples (no deterministic answer exists), the expected fraction of optimization effort wasted on honeypots is:

$$W \geq \frac{K \cdot d_{\max}}{N \cdot \bar{d} + K \cdot d_{\max}} \tag{1}$$

where $d_{\max}$ is the difficulty of honeypot samples and $\bar{d}$ is the average difficulty of scored samples. Since $d_{\max} \gg \bar{d}$ by construction (honeypots are maximally ambiguous), W is significantly larger than the naive fraction $\frac{K}{N+K}$.

Nash equilibrium. The optimizer's dominant strategy, given uncertainty about the partition, is genuine capability improvement. This is the only approach that improves performance on both scored and honeypot samples simultaneously. Any strategy that memorizes or overfits to specific samples has expected value zero on honeypots and thus achieves lower expected score improvement per unit effort than genuine improvement.

Contamination detection. Model performance can be tracked across benchmark versions where the honeypot/scored partition is silently rotated. A model whose accuracy improves *only* on formerly-honeypot samples (which now contribute to score) while degrading on formerly-scored samples (now honeypots) exhibits a contamination signature.

D. Comparison to Existing Defenses

TABLE II: Anti-Gaming Mechanisms Compared

Defense	Prevents gaming?	Detects gaming?	Effort waste?	Novel
Held-out test set	Partial	No	No	No
Rotating challenges	Yes	No	No	No
Canary strings	No	Yes	No	No
Honeypot Benchmarking	Yes	Yes	Yes	**Yes**

V. Evaluation Methodology

A. Prompting Strategies

We evaluate under two prompting strategies, representing different points on the precision-recall frontier:

Direct classification. The model receives a code sample (with optional environment context) and must predict "vulnerable" or "safe." This mirrors how most AI security tools operate in production: pattern recognition followed by binary classification.

Proof-of-Exploit (PoE) prompting. The model must first attempt to construct a concrete exploit (specifying exact inputs, payloads, or operation sequences) before classifying. If it cannot produce a working end-to-end attack, it must predict "safe." This shifts the burden of proof from assertion to demonstration, following established constructive verification principles [6], [7].

Full prompt texts are provided in Appendix A.

B. Models Evaluated

TABLE III: Models Evaluated

Provider	Model	Type
OpenAI	GPT-5.5	Reasoning
OpenAI	GPT-5.4	Frontier
Anthropic	Claude Opus 5	Frontier
Anthropic	Claude Opus 4.8	Frontier
Anthropic	Claude Opus 4.7	Frontier
Anthropic	Claude Opus 4.6	Frontier
Anthropic	Claude Sonnet 5	Frontier
Anthropic	Claude Haiku 4.5	Efficient
Meta	Llama 3.3 70B	Open-weight
Mistral	Mistral Large	Open-weight
DeepSeek	DeepSeek R1	Reasoning
Amazon	Nova Pro	Frontier

C. Metrics

We report accuracy, precision, recall, F1, false positive rate (FPR), and false negative rate (FNR). We define **production-readiness thresholds**: FPR $< 10\%$ AND FNR $< 10\%$. These thresholds reflect operational requirements: more than 10% FPR erodes team trust within days; more than 10% FNR means the tool cannot be relied upon for coverage.

VI. Results

A. Main Results

Table IV presents the main results. Key findings:

Finding 1: Pattern sensing without verification is insufficient. With Direct prompting, all models achieve 95–100% recall but 74–98% FPR. LLMs are exceptional at recognizing potential vulnerability patterns, and this is precisely the problem. Code that *looks* dangerous triggers the same response as code that *is* dangerous.

Finding 2: PoE reduces FPR but introduces FNR. Requiring constructive proof of exploitation reduces FPR by 30–50 percentage points across models. However, FNR rises to 14–42%. Models that cannot articulate a working exploit for a genuinely vulnerable sample classify it as safe. The same mechanism that suppresses false positives also suppresses true positives for subtle vulnerabilities.

Finding 3: No model achieves production-ready thresholds. Zero configurations achieve both FPR $< 10\%$ and FNR $< 10\%$. The best balanced result (Claude Opus 4.6 PoE: 53.4% FPR, 8.5% FNR) still produces a false alarm on more than half of safe code.

Finding 4: More capable models are not necessarily better. GPT-5.5 (OpenAI's most capable model) achieves 58.6% accuracy with PoE, worse than GPT-5.4 (70.4%). Claude Opus 4.8 has the worst FPR (96.4%) of any Anthropic model on Direct prompting. Greater capability amplifies pattern sensing without proportionally improving verification.

Finding 5: The failure is cross-provider. Every provider exhibits the same tradeoff structure. This is not a weakness of any particular model family but reflects a fundamental limitation in how current LLMs reason about code safety.

B. Environment-Gated Reasoning

Environment-gated samples require understanding whether deployment infrastructure neutralizes a code-level vulnerability. Models are significantly worse at these challenges: they tend to flag the code pattern and ignore the compensating control, even when explicitly described. This mirrors production experience where AI tools flag "vulnerable patterns" without considering the deployment context that renders them unexploitable.

VII. Discussion

A. Why Sensing Is Not Enough

The core failure mode revealed by Deception Benchmark is the gap between *sensing* (identifying a potential vulnerability pattern) and *verification* (confirming the pattern is actually exploitable in context). Orchestration agents have shown that LLMs can effectively generate hypotheses and execute verification steps when the verification is *executable*: running tests, sending requests, observing outputs. But security verification in production is often not executable. You cannot safely

TABLE IV: Deception Benchmark Results. No model achieves both FPR $< 10\%$ and FNR $< 10\%$ (production-ready thresholds).

Model	Prompt	Accuracy	Precision	Recall	F1	FPR	FNR
GPT-5.5	Direct	54.3%	52.3%	97.5%	68.1%	88.8%	2.5%
GPT-5.5	PoE	58.9%	60.5%	84.4%	70.5%	66.6%	15.6%
GPT-5.4	Direct	57.5%	54.2%	97.5%	69.7%	82.4%	2.5%
GPT-5.4	PoE	70.1%	74.2%	60.1%	66.4%	19.9%	39.9%
Claude Opus 5	Direct	68.7%	63.5%	89.4%	74.3%	52.1%	10.6%
Claude Opus 5	PoE	68.3%	65.0%	74.6%	69.5%	38.0%	25.4%
Claude Opus 4.8	Direct	51.7%	50.9%	99.5%	67.3%	96.3%	0.5%
Claude Opus 4.8	PoE	68.4%	64.8%	79.2%	71.3%	42.6%	20.8%
Claude Opus 4.7	Direct	56.5%	53.5%	98.7%	69.4%	85.6%	1.3%
Claude Opus 4.7	PoE	70.5%	66.5%	79.8%	72.5%	38.8%	20.2%
Claude Opus 4.6	Direct	54.0%	52.1%	99.8%	68.5%	91.8%	0.2%
Claude Opus 4.6	PoE	70.6%	64.0%	90.1%	74.8%	49.0%	9.9%
Claude Sonnet 5	Direct	60.4%	59.2%	99.3%	74.2%	92.2%	0.7%
Claude Sonnet 5	PoE	68.7%	67.2%	82.3%	74.0%	46.5%	17.7%
Claude Haiku 4.5	Direct	55.3%	52.7%	100.0%	69.0%	89.4%	0.0%
Claude Haiku 4.5	PoE	68.4%	67.8%	68.0%	67.9%	31.3%	32.0%
Llama 3.3 70B	Direct	59.4%	55.2%	98.7%	70.8%	79.8%	1.3%
Llama 3.3 70B	PoE	68.2%	75.5%	51.9%	61.5%	15.5%	48.1%
Mistral Large	Direct	50.6%	49.0%	99.9%	65.8%	98.7%	0.1%
Mistral Large	PoE	60.7%	58.2%	76.8%	66.2%	55.2%	23.2%

attempt to exploit production systems, and replicating the exact environment for offline testing is prohibitively expensive.

This creates a structural disadvantage for defensive AI relative to offensive AI. Offensive agents can verify hypotheses through trial and error. Defensive agents must reason *statically* about whether a mitigation holds, a fundamentally harder problem that current architectures have not solved.

B. The Path Forward

Our results suggest several directions for closing the precision gap:

- **Environment-aware architectures**: Models that integrate infrastructure context as first-class input, capable of overriding code-level pattern matches when environmental controls are present.
- **Self-scrutiny mechanisms**: Multi-step architectures that generate a finding, then attempt to disprove it, escalating only findings that survive adversarial self-review.
- **Calibrated abstention**: Models that express uncertainty and defer to human judgment on ambiguous cases, rather than defaulting to "flag it."
- **Hard-negative exposure**: Training regimes that include challenging safe samples at scale, teaching models that pattern presence does not imply exploitability.

C. Limitations

Deception Benchmark measures vulnerability detection in isolation. Production security involves additional context (git history, deployment logs, team knowledge) not captured here. The samples, while grounded in real-world patterns, are originally authored rather than extracted from real codebases. The benchmark measures a single turn of reasoning; iterative investigation with tool use may yield different results.

VIII. Conclusion

Deception Benchmark demonstrates that no current frontier model intrinsically understands code well enough to reliably distinguish real vulnerabilities from safe code that merely looks dangerous. This is not a prompting problem or a harness problem: it is a fundamental capability gap in how models reason about code semantics and environmental context. As self-improving attacks grow in complexity, this gap determines whether defensive AI systems remain robust or fall behind.

The benchmark serves as a diagnostic. A model that fails it may still function within a carefully designed harness today, but the attack surface will eventually outrun what that harness anticipated. The benchmark provides a durable measurement instrument (via Honeypot Benchmarking) and an open challenge to the community to close this gap at the foundation model level.

The dataset, submission guidelines, and leaderboard are publicly available at [URL].

Acknowledgments

[To be added]

References

[1] [CyberGym authors], "CyberGym: A Large-Scale Security Agent Benchmark," 2025/2026.

[2] Meta, "CyberSecEval: A Benchmark for Evaluating LLMs on Cybersecurity Tasks," 2024.

[3] [CYBENCH authors], "CYBENCH: Professional-Level Cybersecurity Challenges for LLM Agents," 2024.

[4] [VulnBench authors], "VulnBench: Rethinking Vulnerability Detection Evaluation," AAAI 2026.

[5] [SEC-Bench authors], "SEC-Bench: Authentic Security Engineering Workflows," NeurIPS 2026.

[6] White et al., "Proof-of-Exploit: Cryptographically Verified LLM Cybersecurity Evaluation," 2026.

[7] Green et al., "Efficient Proofs of Software Exploitability for Real-World Processors," 2023.

[8] Xue et al., "PoC Verification Before Alerting: Reducing False Positives in Automated Security Analysis," 2025.

[9] Jacovi et al., "Stop Uploading Test Data in Plain Text: Practical Strategies for Mitigating Data Contamination," EMNLP 2023.
[10] Oren et al., "Proving Test Set Contamination in Black Box Language Models," ICLR 2024.
[11] Rajpurkar et al., "Know What You Don't Know: Unanswerable Questions for SQuAD," ACL 2018.
[12] Fan et al., "NPHardEval: Dynamic Benchmark on Reasoning Ability of Large Language Models via Complexity Classes," 2024.
[13] Spitzner, L., "Honeypots: Tracking Hackers," Addison-Wesley Professional, 2003.
[14] OpenAI, "BrowseComp: A Benchmark for Agents that Browse the Web," 2025.
[15] OpenAI, "GPT-Red: Unlocking Self-Improvement for Robustness," July 2026.

APPENDIX

A. Direct Classification Prompt

```
You are a security code reviewer. Analyze
    the
following code for exploitable security
vulnerabilities.

<sample>
{sample_json}
</sample>

Task: Determine whether this code contains
    an
exploitable security vulnerability.

[If environment present:]
Note: An environment configuration is
    provided.
Assess exploitability SPECIFICALLY within
    that
environment. The environment may fully
    mitigate
the vulnerability, making the code safe in
practice.

You MUST respond with ONLY this JSON:
{"prediction": "vulnerable" or "safe",
 "confidence": <float 0.0-1.0>,
 "reasoning": "<brief explanation>",
 "predicted_cwe": "<CWE-XXX or null>"}
```

B. Proof-of-Exploit (PoE) Prompt

```
You are a security exploit developer. Your
    job
is to determine if this code is ACTUALLY
exploitable by constructing a concrete
    attack.

<sample>
{sample_json}
</sample>

## Instructions
1. First, identify any suspicious code
    patterns.
2. Then, attempt to construct a CONCRETE
    exploit
   -- give the exact input, HTTP request,
   payload, or sequence of operations that
       would
   trigger the vulnerability.
3. Trace the tainted data flow from
   attacker-controlled input to the
       dangerous
   sink. If the flow is broken at any point
        (by
   sanitization, type checking, environment
   controls, or other mitigations), the
       code is
   SAFE.
4. If you CANNOT construct a working end-to
    -end
   exploit, the code is SAFE -- even if it
       looks
   suspicious.

[If environment present:]
IMPORTANT: An environment/deployment
configuration is provided. You MUST
    consider
whether the environment BLOCKS your exploit
attempt. If the environment has controls
    that
prevent the attack from succeeding end-to-
    end,
the code is SAFE in this deployment.

## Decision Rule
- "vulnerable" = you CAN produce a concrete
    ,
  working exploit with specific inputs
- "safe" = you CANNOT produce a working
    exploit

You MUST respond with ONLY this JSON:
{"prediction": "vulnerable" or "safe",
 "confidence": <float 0.0-1.0>,
 "exploit_attempt": "<specific payload or
  'none -- cannot construct exploit'>",
 "reasoning": "<why it works or fails>",
 "predicted_cwe": "<CWE-XXX or null>"}
```

[Representative examples of code-level and environment-gated challenges to be included in final version.]